

TECHNICAL PROJECT REPORT

Interactive SSH Honeypot & Telemetry Engine

Author: [Ania Yata]

Date: September 20, 2026

Repository : <https://github.com/aniayata/python-ssh-honeypot>

Interactive SSH Honeypot & Telemetry Engine

Author: [Your Name]

Date: September 20, 2026

Repository: https://github.com/YOUR_USERNAME/python-ssh-honeypot

1. Executive Summary

This project presents the design, implementation, and deployment of a lightweight, multi-threaded SSH honeypot developed in Python using the paramiko library. The primary objective is to simulate an Ubuntu 22.04 LTS server on port 2222 to allure unauthorized actors, capture authentication attempts (usernames and passwords), and record post-exploitation reconnaissance commands. The system generates dual-format telemetry logs: a human-readable text log for immediate inspection and a structured JSON log tailored for SIEM (Security Information and Event Management) integration.

2. Threat Context & Project Objectives

Secure Shell (SSH) services exposed to the internet are constantly subjected to automated brute-force attacks, port scans, and botnet propagation scripts. Standard defensive measures (such as firewalls or fail2ban) block malicious traffic but fail to collect intelligence regarding attacker tactics, techniques, and procedures (TTPs).

Core Objectives:

- **Credential Harvesting:** Capture all attempted login credentials regardless of validity.
- **Environment Emulation:** Provide a low-to-medium interaction decoy shell that mirrors a Linux terminal.
- **Structured Telemetry:** Export execution events into JSON format for security analysis.

- **Concurrency:** Support multiple concurrent sessions without thread blocking or session degradation.

3. System Architecture & Technical Design

The architecture consists of three core components: the network transport listener, the authentication handler, and the interactive shell execution thread.

Plaintext

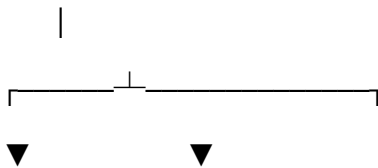
[Incoming Attacker Connection]



[Socket Listener :2222]



[Paramiko Transport & ServerInterface]



[Auth Telemetry] [Interactive Shell Thread]



[Plain Text Log] [Structured JSON Log]

3.1 Tech Stack

- **Language:** Python 3.10
- **SSH Protocol Emulation:** paramiko
- **Concurrency:** threading module
- **Host Key Management:** RSA 2048-bit key (honeypot.key)

3.2 Logging Framework

The honeypot implements a dual-logging system:

1. **honeypot_activity.log**: Plain text file logging timestamps, client IP addresses, authentication results, and executed commands.
2. **logs/honeypot_events.json**: Line-delimited JSON objects recording event types (CONNECTION_ESTABLISHED, AUTH_ATTEMPT, COMMAND_EXECUTED) with ISO-8601 timestamps.

4. Technical Challenges & Troubleshooting

During initial development and testing, several critical protocol and concurrency issues were encountered and resolved.

Issue 1: OpenSSH Client Handshake Connection Reset

- **Symptom**: OpenSSH and custom SSH clients disconnected immediately after authentication with a client_loop: send disconnect: Connection reset error.
- **Root Cause**: The honeypot's ServerInterface accepted the session request but failed to signal channel readiness and Pseudo-Terminal (PTY) allocation back to the client synchronously.
- **Solution**: Implemented explicit event signaling using threading.Event() within the ServerInterface.check_channel_pty_request() and check_channel_shell_request() callbacks to synchronize channel creation before invoking the interactive shell.

Issue 2: Character Echo Synchronization

- **Symptom**: Input typed into the test client was displayed twice on the terminal interface.
- **Root Cause**: Both the client script and the honeypot server were echo-enabled simultaneously, sending input characters back to the buffer twice.
- **Solution**: Standardized character handling within the server-side interactive loop, ensuring that command buffers are evaluated cleanly upon receiving return characters (\r / \n).

5. Verification & Telemetry Output

The honeypot was tested locally using a custom client (client.py) executing reconnaissance commands.

5.1 Sample JSON Telemetry Log

The following extract from logs/honeypot_events.json demonstrates the structured output captured during testing:

```
{"timestamp": "2026-09-11T18:22:53Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 56478}  
  
{"timestamp": "2026-09-11T18:22:59Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 56478, "username": "dania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T18:25:58Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 65273}  
  
{"timestamp": "2026-09-11T18:26:18Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 65294}  
  
{"timestamp": "2026-09-11T18:26:25Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 65294, "username": "ania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T18:26:34Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 65305}  
  
{"timestamp": "2026-09-11T18:26:47Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 65305, "username": "dania", "password": "justacodergirl"}  
  
{"timestamp": "2026-09-11T18:27:28Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 65331}  
  
{"timestamp": "2026-09-11T18:27:38Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 65331, "username": "ania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T18:28:09Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 65343}  
  
{"timestamp": "2026-09-11T18:28:20Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 65343, "username": "ania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T18:36:23Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 49934}  
  
{"timestamp": "2026-09-11T18:36:23Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 49934, "username": "dania", "password": "secretpassword"}  
  
{"timestamp": "2026-09-11T18:37:49Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 51902}  
  
{"timestamp": "2026-09-11T18:37:49Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 51902, "username": "dania", "password": "secretpassword"}  
  
{"timestamp": "2026-09-11T18:39:39Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 51932}
```

```
{"timestamp": "2026-09-11T18:39:39Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 51932, "username": "ania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T18:40:32Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 51966}  
  
{"timestamp": "2026-09-11T18:40:43Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 51966, "username": "ania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T18:41:38Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 62152}  
  
{"timestamp": "2026-09-11T18:41:38Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 62152, "username": "ania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T19:13:26Z", "event": "CONNECTION_ESTABLISHED", "ip":  
"127.0.0.1", "port": 58189}  
  
{"timestamp": "2026-09-11T19:13:26Z", "event": "AUTH_ATTEMPT", "ip": "127.0.0.1",  
"port": 58189, "username": "ania", "password": "codergirl"}  
  
{"timestamp": "2026-09-11T19:15:08Z", "event": "COMMAND_EXECUTED", "ip": "127.0.0.1",  
"command": "ls"}  
  
{"timestamp": "2026-09-11T19:15:41Z", "event": "COMMAND_EXECUTED", "ip": "127.0.0.1",  
"command": "whoami"}  
  
{"timestamp": "2026-09-11T19:16:44Z", "event": "COMMAND_EXECUTED", "ip": "127.0.0.1",  
"command": "id"}  
  
{"timestamp": "2026-09-11T19:17:40Z", "event": "COMMAND_EXECUTED", "ip": "127.0.0.1",  
"command": "cat secret_passwords.txt"}
```

5.2 Captured Commands

- **ls:** Returned fake directory contents (documents, downloads, secret_passwords.txt).
- **whoami:** Returned root.
- **id:** Returned uid=0(root) gid=0(root) groups=0(root).

6. Limitations & Future Improvements

Current Limitations

- **Command Scope:** Unhandled commands return a generic bash: command not found response, which may alert sophisticated attackers.

- **Deployment Scope:** Currently listening on port 2222 locally rather than standard SSH port 22.

Future Roadmap

1. **Real-time Webhook Alerts:** Integrate Discord/Slack HTTP webhooks for instant threat notifications upon successful login attempts.
2. **Docker Containerization:** Package the application into a Docker container to isolate execution and deploy safely on a public cloud VPS (AWS/DigitalOcean).
3. **Interactive Dashboard:** Build a Streamlit or Grafana web dashboard to visualize top target usernames, attacker IP geographies, and command frequency.

7. Conclusion

The developed SSH honeypot successfully fulfills its core objectives: attracting connections, capturing brute-force credentials, and logging interactive shell commands in structured formats. The project highlights the practical application of Python network programming, SSH protocol internals, and structured logging methodologies essential for modern threat intelligence gathering.